

L8 Colab 天气分析综合练习题集

这两道题继续在 **Google Colab** 上完成。操作步骤：

1. 打开 [Google Colab](#)
2. 新建一个 Notebook
3. 把数据文件上传到 Colab（点击左侧文件夹图标，然后上传）
4. 在代码格里写代码，运行看结果

完成后把你的 **Colab 链接** 放到 `submission` 文件夹下。

数据文件还是 `climate_data_2017.csv`，13 列。这次主要用到：

- `date`（日期，格式 `2017-01-03`）
- `state`（州名）
- `rainfall`（降雨量，毫米）
- `temp9am`（上午 9 点温度）
- `humidity9am`（上午 9 点湿度）

Exercise 1: 各州平均降雨量 ★★

计算每个州的平均降雨量，按州名字母顺序输出。

这道题要求用**两种方法**完成：先用 Python 原生的文件读取（`with open`），再用 pandas 的 `groupby` 实现同样的效果。

两种方法的输出完全一样——做两遍是为了让你理解：pandas 的一行代码背后，其实就是 Method 1 里那个循环在做的事。

Method 1: 使用 Python 原生方法 `with open` 读取

手动逐行处理：跳过标题行，用 `split(',')` 拆字段，用字典收集每个州的所有降雨量，最后手动算平均值。

这就是 L3/L4 一直在练的老套路。

要求：

1. 用 `with open("climate_data_2017.csv") as f` 打开文件
2. 用 `next(f)` 跳过第一行标题
3. 每一行用 `strip()` 去掉末尾换行，再用 `split(",")` 拆字段
4. 取出 `state`（第 2 列，下标 1）和 `rainfall`（第 7 列，下标 6）
5. 把 `rainfall` 转成 `float`

6. 用一个字典，key 是州名，value 是该州所有降雨量的列表
7. 如果这个州第一次出现，先创建空列表；否则往列表里 `append`
8. 遍历字典，对每个州计算平均值：`sum(列表) / len(列表)`
9. 按州名字母顺序输出：`州名 : 平均降雨量`
10. 平均降雨量保留 1 位小数

测试代码 + 预期输出

```
# NSW : 2.1
# QLD : 5.1
# SA : 2.8
# TAS : 2.5
# VIC : 2.7
# WA : 7.8
```

代码框架

```
rain_by_state = {}

with open("climate_data_2017.csv") as f:
    next(f) # 跳过标题行
    for row in f:
        values = row.strip().split(",")

        # 取出 state 和 rainfall
        state = None
        rainfall = None

        # 如果这个州第一次出现，先创建空列表
        # 否则把 rainfall 追加进去
        pass

# 按州名字母顺序输出
for state in sorted(rain_by_state):
    rain_list = rain_by_state[state]
    avg = sum(rain_list) / len(rain_list)
    print(state, ":", round(avg, 1))
```

Method 2: 使用 pandas 的 `groupby` 实现

同样的结果，用 pandas 只需要三步：`read_csv` → `groupby` → `.mean()`。

要求：

1. 导入 `pandas as pd`
2. 用 `pd.read_csv()` 读取文件
3. 用 `state` 分组
4. 对 `rainfall` 求 `.mean()`
5. 按州名字母顺序输出： `州名 : 平均降雨量`
6. 保留 1 位小数

测试代码 + 预期输出

```
# NSW : 2.1
# QLD : 5.1
# SA : 2.8
# TAS : 2.5
# VIC : 2.7
# WA : 7.8
```

代码框架

```
import pandas as pd

df = pd.read_csv("climate_data_2017.csv")

# 按 state 分组, 对 rainfall 求平均值
avg_rain = None

# 按州名字母顺序输出
for state in sorted(avg_rain.index):
    print(state, ":", round(float(avg_rain[state]), 1))
```

对比两种方法

做完之后，看看你的两段代码：

	Method 1 with open	Method 2 pandas
读文件	手动 <code>open</code> 、 <code>next(f)</code> 跳标题、 <code>split</code> 拆字段	<code>pd.read_csv()</code> 一行搞定
分组	自己用字典， <code>if state not in dict</code> 判断	<code>groupby('state')</code>
求平均	<code>sum(list) / len(list)</code>	<code>.mean()</code>
排序	<code>sorted(dict)</code>	<code>sorted(series.index)</code>
代码行数	~15 行	~5 行

Method 1 虽然长，但每一步你都能看到——数据怎么来的、字典长什么样、平均值怎么算的。Method 2 简洁但把这些细节"藏"在函数里了。两种都要会，才算真正懂数据分析。

Exercise 2: 各州高湿天的统计 ★★★★★

定义：如果某天某个城市的 `humidity9am > 80`，就算这个州这天有一条"高湿记录"。

你需要：

1. 先筛选出所有湿度大于 80 的记录
2. 按 `state` 分组，统计每个州有多少条高湿记录（天数）
3. 按 `state` 分组，算出每个州高湿天的平均温度（`temp9am` 的平均值）
4. 有些州可能一条高湿记录都没有——这些州也要输出，天数显示 `0`，温度显示 `N/A`

这题综合了前面学过的：布尔筛选（L5/L6）、`groupby` + 聚合（L7）、遍历时处理缺失（L6 的 `.get()`）。

要求：

1. 导入 `pandas as pd`
2. 读取 `climate_data_2017.csv`
3. 获取数据中所有州名（用于最终输出，保证不遗漏）
4. 用 `humidity9am > 80` 做布尔筛选，拿到高湿记录
5. 对高湿记录按 `state` 分组，用 `.size()` 统计每州条数
6. 对高湿记录按 `state` 分组，用 `.mean()` 算每州 `temp9am` 的平均值
7. 把两个结果都转成字典
8. 遍历所有州名（字母顺序），用 `.get()` 取值——取不到就说明该州没有高湿天
9. 输出格式：州名 : X days, avg temp Y（天数是整数，温度保留 1 位小数）
10. 如果某州 0 天，温度写 `N/A`

测试代码 + 预期输出

```
# NSW : 4 days, avg temp 21.0
# QLD : 3 days, avg temp 25.9
# SA : 0 days, avg temp N/A
# TAS : 6 days, avg temp 13.0
# VIC : 2 days, avg temp 15.9
# WA : 0 days, avg temp N/A
```

代码框架

```
import pandas as pd

df = pd.read_csv("climate_data_2017.csv")

# 获取所有州名，排好序
all_states = sorted(df["state"].unique())

# 布尔筛选: 哪些行的 humidity9am > 80
humid_days = None

# 对高湿记录按州分组，统计条数
count_dict = None

# 对高湿记录按州分组，算平均温度
temp_dict = None

# 遍历所有州名，输出
for state in all_states:
    count = count_dict.get(state, 0)
    if count > 0:
        avg_temp = round(temp_dict[state], 1)
        print(f"{state} : {count} days, avg temp {avg_temp}")
    else:
        print(f"{state} : 0 days, avg temp N/A")
```

这题在练什么

1. 布尔筛选是 pandas 最核心的操作——先"缩小范围"，再做后续统计
2. 同一个筛选结果可以做多种聚合（`.size()` 数个数、`.mean()` 算平均）
3. 不是所有分组都会出现——要用 `.get(state, 0)` 或 `if` 来兜底
4. 这就是真实数据分析的常见场景：先过滤，再分组统计，最后输出时处理缺失

提交说明

完成后，在 `submission` 文件夹下新建一个文件（比如 `你的名字_L8.txt`），把你的 Colab Notebook 链接粘贴进去。

Exercise 3 : LeetCode 练习：滑动窗口

完成 小而美的算法技巧：前缀和数组 的阅读

<https://labuladong.online/zh/algo/data-structure/prefix-sum/>

在 LeetCode 上完成以下两个练习，并截图提交：

303. 区域和检索 - 数组不可变

304. Range Sum Query 2D - Immutable